

Nix and NixOS Syllabus 2026-07-21

1954

This is a fantastic initiative. Transitioning from traditional imperative Linux administration (like RHEL/Ubuntu) to the declarative, functional paradigm of Nix and NixOS requires a fundamental shift in mindset.

To build a course equivalent to an enterprise RHCE (Red Hat Certified Engineer) but tailored for the modern Nix ecosystem (incorporating the community consensus, best practices, and tools popularized on GitHub, NixOS Discourse, and Twitter/X up to 2026), we must cover everything from the raw Nix language to multi-node enterprise deployments.

Here is the comprehensive, professional-grade syllabus for "**Mastering Nix & NixOS: From Zero to Enterprise Production.**"

Course Overview: Mastering Nix & NixOS

Level: Beginner to Enterprise/Production Architect

Paradigm: Declarative, Immutable, Reproducible Infrastructure

Modern Stack Focus: Nix Flakes, Home Manager, Disko, Colmena, Lanzaboote, and NixOS Module System.

Target Audience

- System Administrators and DevOps Engineers transitioning from traditional Linux.

- Software Developers wanting reproducible, hermetic development environments.
 - Infrastructure Architects looking to implement Immutable Infrastructure and GitOps.
-

Phase 1: The Foundations & The Nix Language

Before touching the OS, you must understand the package manager and the language that drives it.

Module 1: The Nix Philosophy & Architecture

- **Objective:** Understand *why* Nix exists and how its underlying architecture solves dependency hell and system rot.
- **Topics:**
 - Imperative vs. Declarative vs. Functional system management.
 - The Nix Store (`/nix/store`): Paths, hashes, and content-addressable storage.
 - Garbage Collection (GC): Roots, profiles, and `nix-collect-garbage` .
 - The Build Sandbox: Hermetic builds and isolation.
 - Nixpkgs: The largest, most up-to-date monorepo in the world.
- **Key Concepts:** Derivations, Store Paths, GC Roots, Symlinks.

Module 2: The Nix Expression Language

- **Objective:** Master the custom, lazy, purely functional programming language used to define packages and configurations.
- **Topics:**
 - Basic Data Types: Strings, integers, booleans, paths.
 - Lists and Attribute Sets (Attrsets): The core data structures.

- **Functions:** Syntax, arguments, default values, and `@` pattern matching.
 - **Control Flow & Scoping:** `let ... in`, `with`, `rec`, `if ... then ... else`.
 - **Imports and Modules:** `import`, `builtins`, and evaluating expressions.
 - **Debugging:** `builtins.trace`, `seq`, and `deepSeq`.
 - **Key Concepts:** Lazy evaluation, Attrset merging, Higher-order functions.
-

Phase 2: The Modern Nix Workflow (Flakes)

Moving away from legacy `nix-env` and channels to the modern, locked, and reproducible standard.

Module 3: Nix Flakes & The Modern CLI

- **Objective:** Master `nix` (the new CLI) and Flakes, the standard for reproducible projects and system configurations.
 - **Topics:**
 - Anatomy of a `flake.nix`: `inputs`, `outputs`, and `systems`.
 - Flake references and URLs (GitHub, GitLab, local paths).
 - The Flake Lock file (`flake.lock`): Ensuring reproducibility.
 - `nix run`, `nix build`, and `nix shell`.
 - Updating inputs and managing flake lifecycle.
 - **Key Concepts:** Flake schemas, `devShells`, `packages`, `nixosConfigurations`.
-

Phase 3: NixOS Core & System Administration

Transitioning from using Nix as a package manager to using NixOS as an operating system.

Module 4: NixOS Core Configuration

- **Objective:** Learn to configure the OS declaratively using the NixOS Module System.
- **Topics:**
 - The NixOS Module System: `config`, `options`, `mkOption`, `mkDefault`, `mkForce`, `mkMerge`.
 - Anatomy of `configuration.nix` and `hardware-configuration.nix`.
 - Managing Users, Groups, and Sudoers.
 - Boot management: `systemd-boot` vs. GRUB, kernel selection, and `sysctl` parameters.
 - Timezone, Locale, and Console configuration.
 - Rebuild strategies: `switch`, `boot`, `test`, `dry-activate`.
- **Key Concepts:** Module evaluation, type checking, option definitions.

Module 5: Advanced NixOS System Administration

- **Objective:** Handle complex system administration tasks the "NixOS way".
- **Topics:**
 - Networking: `systemd-networkd`, `NetworkManager`, wireless (`iwd/wpa_supplicant`), and firewall (`nftables`).
 - Services: Writing custom `systemd` services and timers in Nix.
 - Secrets Management: Integrating `agenix` or `sops-nix` for decrypting secrets at runtime.
 - Virtualization & Containers: NixOS containers, LXD, Docker, and Podman.
 - NixOS Containers vs. LXC vs. VMs.

- **Key Concepts:** Systemd integration, secret injection, network namespaces.

Module 6: Declarative Storage & Disk Management

- **Objective:** Master modern, declarative disk partitioning and filesystem management (a major pain point for beginners).
 - **Topics:**
 - Introduction to **Disko**: Declarative disk partitioning.
 - Filesystems: Ext4, BTRFS (subvolumes, snapshots), and ZFS (pools, datasets).
 - LVM and RAID configurations via Disko.
 - Swap management and tmpfs configurations.
 - Automating disk setup for cloud and bare-metal provisioning.
 - **Key Concepts:** Disko schemas, BTRFS/ZFS integration, fstab generation.
-

Phase 4: The User Environment & Development

Managing the user space, dotfiles, and creating hermetic development environments.

Module 7: Home Manager & User Configurations

- **Objective:** Manage user-specific configurations, dotfiles, and packages independently of the system root.
- **Topics:**
 - Home Manager architecture: Standalone vs. NixOS Module.
 - Managing dotfiles: `home.file`, `xdg.configFile`, and `home.activation` scripts.

- Configuring programs declaratively (Git, Neovim, Alacritty, Wayland/X11).
- Managing user-level systemd services.
- Standalone Home Manager on non-NixOS systems (Ubuntu, macOS).
- **Key Concepts:** User profiles, XDG base directory, activation scripts.

Module 8: Hermetic Development Environments

- **Objective:** Replace Docker/Vagrant for local dev with instant, reproducible Nix dev shells.
 - **Topics:**
 - `mkShell` and `buildInputs` vs `nativeBuildInputs`.
 - Environment variables: `shellHook`, `LD_LIBRARY_PATH`, `PKG_CONFIG_PATH`.
 - **Direnv & nix-direnv:** Automatic shell loading upon `cd`.
 - Language-specific tooling:
 - Python (`python.withPackages` , `venvShellHook`)
 - Rust (`rustPlatform.buildRustPackage` , `crane`)
 - Node.js (`buildNpmPackage` , `pnpm.fetchDeps`)
 - Go & C/C++ (`buildGoModule` , `stdenv`)
 - **Key Concepts:** Devshells, flake `devShells`, language ecosystems.
-

Phase 5: Packaging & The Nixpkgs Ecosystem

How to package software that isn't in the official repository, and how to contribute.

Module 9: Writing Custom Packages (Derivations)

- **Objective:** Learn the dark art of writing Nix derivations to build software from source.
- **Topics:**
 - `stdenv.mkDerivation`: The phases (unpack, patch, configure, build, install, fixup).
 - Fetchers: `fetchurl`, `fetchFromGitHub`, `fetchgit`, `fetchpatch`.
 - Handling dependencies and build inputs.
 - Fixing common build failures: Patching source code, fixing interpreter paths (`patchShebangs`), handling RPATH.
 - Meta attributes: licenses, maintainers, descriptions.
- **Key Concepts:** Derivation phases, `stdenv`, patching, fixup hooks.

Module 10: Overlays, Custom Repos, and `callPackage`

- **Objective:** Modify existing packages and structure your own Nix code professionally.
 - **Topics:**
 - The `callPackage` pattern: Dependency injection in Nix.
 - Overlays: Modifying `nixpkgs` without forking (`self: super: ...`).
 - Creating custom package sets and local overlays.
 - Using `npins` or `flake-compat` for non-flake legacy projects.
 - Contributing to Nixpkgs: PR workflow, ofborg, and review processes.
 - **Key Concepts:** Overlays, `callPackage`, package sets, Nixpkgs contribution guidelines.
-

Phase 6: Enterprise Infrastructure, CI/CD & Production

Scaling NixOS across multiple machines, cloud providers, and integrating with enterprise workflows.

Module 11: Multi-Machine Management & GitOps

- **Objective:** Deploy and manage fleets of NixOS machines declaratively.
- **Topics:**
 - Introduction to **Colmena** (and Morph/NixOps): Hive configurations, deployment targets.
 - Building and deploying to remote hosts via SSH.
 - NixOS in the Cloud: AWS (AMIs), GCP, Hetzner, and DigitalOcean.
 - Using `nixos-generators` to build custom ISOs, cloud images, and SD cards.
 - Managing infrastructure as code (IaC) with NixOS and Terraform/Ansible bridges.
- **Key Concepts:** Colmena hives, remote deployment, image generation.

Module 12: CI/CD, Binary Caching, and Production Hardening

- **Objective:** Optimize build times, secure the OS, and ensure enterprise-grade reliability.
- **Topics:**
 - **Binary Caches:** Using Cachix, Garnix, or setting up a self-hosted binary cache.
 - CI/CD Integration: Running Nix in GitHub Actions, GitLab CI (using `DeterminateSystems/nix-installer`).
 - **Security & Hardening:**
 - Secure Boot with **Lanzaboote** (systemd-boot stub).
 - Full Disk Encryption (LUKS) with TPM2 auto-unlock (Clevis).
 - AppArmor/SELinux profiles and sandboxing (`nixpak`, `bwrap`).

- **Monitoring & Observability:** Declarative Prometheus, Grafana, and Loki setups.
 - **Disaster Recovery:** Backup strategies (Borg, Restic, ZFS snapshots) configured via Nix.
 - **Key Concepts:** Binary substitutes, Secure Boot, TPM2, CI/CD pipelines, immutable infrastructure.
-

Phase 7: Capstone Project

- **Objective:** Prove mastery by building a complete, production-ready environment.
 - **Project:** "The Immutable Homelab/Cloud Cluster"
 - Deploy 3 NixOS nodes (1 Controller, 2 Workers) using **Colmena** and **Disko**.
 - Nodes must be fully encrypted (LUKS + TPM2) and boot with Secure Boot (**Lanzaboote**).
 - Secrets managed via **sops-nix**.
 - User environments managed via **Home Manager**.
 - A CI/CD pipeline (GitHub Actions) that builds the configuration, pushes to a **Cachix** binary cache, and triggers a Colmena deployment.
 - A declarative monitoring stack (Prometheus/Grafana) to observe the fleet.
-

How we will proceed:

This syllabus is designed to be exhaustive. It takes the scattered knowledge from the NixOS Discourse, GitHub issues, and community blogs from the last few years and structures it into a logical, enterprise-grade learning path.

Next Steps:

- .. Review the syllabus. Are there any specific areas (e.g., more focus on cloud, more focus on desktop/Wayland, or deeper dive into the Nix language) you want to add or modify?
- !. Once you are happy with this structure, **tell me to begin**, and we will start generating **Module 1**, complete with theoretical explanations, code snippets, and **Practical Workbook Labs** (step-by-step exercises).

Creating a **comprehensive, professional-level syllabus for Nix and NixOS**—akin to RHEL or other enterprise Linux courses—requires a structured approach. This syllabus will cover **foundational concepts, intermediate skills, advanced topics, and production-grade deployment strategies**, along with **hands-on labs, real-world scenarios, and best practices** gathered from the Nix/NixOS ecosystem (blogs, forums, documentation, and community discussions).

I'll break this into **modules**, each with **theoretical lessons, practical labs, and real-world use cases**. The goal is to make this a **single-point reference** for mastering Nix and NixOS from scratch to production.

Proposed Syllabus Structure

(Modular, Progressive, and Hands-On)

Module 1: Introduction to Nix and NixOS

Objective: Understand the philosophy, architecture, and advantages of Nix and NixOS.

1.1 What is Nix?

- The Nix Package Manager: Immutable, Declarative, Reproducible
- Comparison with Traditional Package Managers (APT, YUM, Pacman)
- Nix as a **Purely Functional** Package Manager
- **Nix Store**: Immutable, Hash-Based, and Isolated
- **Nix Language**: A Domain-Specific Language (DSL) for Configurations

1.2 What is NixOS?

- NixOS as a **Declarative Linux Distribution**
- Immutable Infrastructure: No In-Place Upgrades
- **Atomic Upgrades and Rollbacks**
- Comparison with Traditional Linux Distributions (RHEL, Debian, Arch)

1.3 Why Nix/NixOS?

- **Reproducibility**: Same Configuration = Same System
- **Reliability**: No Dependency Hell, No Breaking Changes
- **Security**: Immutable, Isolated Environments
- **DevOps & CI/CD**: Perfect for Infrastructure as Code (IaC)
- **Use Cases**: Development, Servers, Containers, Edge Computing

1.4 Nix/NixOS Ecosystem

- **Nixpkgs**: The Largest Package Repository
- **NixOS Modules System**
- **Home Manager**: User-Level Declarative Configurations
- **Nix Flakes**: Reproducible Development Environments
- **Nix in Industry**: Companies Using Nix (e.g., Tweag, IOHK, Target, etc.)

1.5 Setting Up the Learning Environment

- Installing Nix on Linux/macOS/Windows (WSL)
- Installing NixOS in a VM (QEMU/VirtualBox)
- NixOS Live ISO for Testing

- **Recommended Tools:** `nix-shell` , `nix-build` , `nixos-rebuild`
-

Practical Lab 1: Installing Nix & NixOS

- Install Nix on your primary OS.
 - Set up a NixOS VM using QEMU.
 - Verify Nix and NixOS installations.
 - Explore the Nix store (`/nix/store`).
-

Module 2: Nix Package Manager Deep Dive

Objective: Master Nix as a package manager for development and system management.

2.1 Nix Basics

- **Nix Expressions:** The Nix Language Syntax
- **Derivations:** The Core of Nix Packages
- **Nix Store Paths:** `/nix/store/<hash>--<name>`
- **Nix Profiles:** User-Specific Environments
- **Nix Channels:** Tracking Stable vs. Unstable Packages

2.2 Working with Nix Packages

- **Searching Packages:** `nix search`
- **Installing Packages:** `nix-env -i`
- **Upgrading Packages:** `nix-env -u`
- **Removing Packages:** `nix-env -e`
- **Listing Installed Packages:** `nix-env -q`

2.3 Nix Shells for Development

- **Ad-Hoc Shells:** `nix-shell -p <package>`

- **Declarative Shells:** `shell.nix` Files
- Using `nix-shell` for Project-Specific Dependencies
- Temporary vs. Permanent Shells

2.4 Nix Garbage Collection

- **Manual GC:** `nix-collect-garbage`
- **Automatic GC:** `nix.gc` Options
- **Generational GC:** Keeping Multiple Versions
- **Store Optimization:** `nix-store --optimise`

2.5 Nix and Multi-User Systems

- **Nix Daemon** (`nix-daemon`)
- **User Permissions and Nix Store Access**
- **Nix in Multi-User Environments** (e.g., Servers)

Practical Lab 2: Nix Package Management

- Install, upgrade, and remove packages.
- Create a `shell.nix` for a Python/Node.js project.
- Use `nix-shell` to manage project dependencies.
- Experiment with garbage collection.

Module 3: Nix Language & Custom Packages

Objective: Learn the Nix language to write custom derivations and packages.

3.1 Nix Language Fundamentals

- **Basic Syntax:** Attributes, Lists, Strings, Functions
- **Imports:** Reusing Nix Expressions

- **Conditionals:** `if-then-else`
- **Loops:** `builtins.map` , `builtins.fold` , `rec`
- **Built-in Functions:** `builtins.attrNames` , `builtins.filter` , etc.

3.2 Writing Custom Derivations

- **Derivation Basics:** `stdenv.mkDerivation`
- **Inputs, Build Phases, and Outputs**
- **Overriding Packages:** `overrideAttrs`
- **Custom Build Scripts:** `buildPhase` , `installPhase`

3.3 Packaging Software for Nix

- **Packaging a Simple CLI Tool**
- **Packaging a Python/Node.js/Rust Project**
- **Handling Dependencies**
- **Cross-Platform Considerations**

3.4 Nixpkgs Contributions

- **Forking Nixpkgs**
- **Adding a Package to Nixpkgs**
- **Submitting a PR to Nixpkgs**
- **Nixpkgs Overlays:** Custom Package Repositories

Practical Lab 3: Writing Custom Nix Packages

- Write a derivation for a simple CLI tool.
 - Package a Python project with dependencies.
 - Create a custom overlay for internal use.
-

Module 4: NixOS Configuration & Deployment

Objective: Learn to configure NixOS declaratively for servers and workstations.

4.1 NixOS Configuration Basics

- `configuration.nix` : The Main Configuration File
- **NixOS Modules System:** Reusable Configurations
- **Options:** `services.` , `programs.` , `users.` , `fileSystems.`
- **Enable/Disable Services:** `services.nginx.enable = true;`

4.2 Managing Services

- **Systemd Services in NixOS**
- **Custom Services:** `systemd.services.<name>`
- **Supervisord, Docker, and Containerized Services**
- **Logging & Monitoring:** `services.logrotate` , `services.prometheus`

4.3 Networking in NixOS

- **Static/DHCP Networking**
- **Firewall (nftables / iptables)**
- **SSH, VPN (WireGuard/OpenVPN)**
- **DNS (dnsmasq , bind)**
- **NixOS as a Router**

4.4 Storage & Filesystems

- **Partitioning & Formatting**
- **LUKS Encryption**
- **LVM, RAID, Btrfs, ZFS**
- **Automounting Filesystems**
- **NixOS on NFS/iSCSI**

4.5 Users & Permissions

- **User Management:** `users.users.<name>`

- Groups & Sudo Access
- Home Directories & Skeletons
- PAM & Authentication

4.6 NixOS on Bare Metal & Cloud

- Installing NixOS on Physical Hardware
 - NixOS on Cloud Providers (AWS, GCP, Azure)
 - NixOS on Proxmox/ESXi
 - NixOS Containers (LXC, Docker, Podman)
-

Practical Lab 4: Configuring NixOS

- Set up a basic NixOS VM with SSH, Nginx, and a user.
 - Configure a static IP and firewall.
 - Deploy a NixOS system on a cloud provider (e.g., AWS).
 - Encrypt a partition with LUKS.
-
-

Module 5: Advanced NixOS Features

Objective: Explore advanced NixOS features for production environments.

5.1 NixOS Modules & Custom Modules

- Writing Reusable NixOS Modules
- Module Arguments & Defaults
- Module Imports & Overrides
- Testing Modules with `nixos-test`

5.2 NixOS and Containers

- Docker & Podman in NixOS

- **NixOS as a Docker Host**
- **Building Custom Docker Images with Nix**
- **Kubernetes & NixOS**

5.3 NixOS and Virtualization

- **QEMU/KVM Virtualization**
- **Libvirt & Virt-Manager**
- **NixOS as a Hypervisor**
- **Cloud-Init & Cloud Images**

5.4 NixOS and Security

- **Hardening NixOS: SELinux, AppArmor, Grsecurity**
- **Mandatory Access Control (MAC)**
- **Secure Boot & TPM**
- **NixOS for Air-Gapped Systems**

5.5 NixOS and High Availability

- **Clustering with NixOS**
- **Load Balancing (HAProxy, Nginx)**
- **Database Clustering (PostgreSQL, MySQL)**
- **Distributed Storage (Ceph, GlusterFS)**

Practical Lab 5: Advanced NixOS Deployments

- **Create a custom NixOS module for a service.**
 - **Deploy a Dockerized application on NixOS.**
 - **Set up a KVM-based VM with NixOS.**
 - **Harden a NixOS system with SELinux.**
-

Module 6: Nix Flakes & Reproducible Environments

Objective: Master Nix Flakes for reproducible development and deployment.

6.1 Introduction to Flakes

- What are Flakes?
- Flakes vs. Traditional Nix Workflows
- Flake Inputs, Outputs, and Lock Files

6.2 Using Flakes

- Enabling Flakes in Nix/NixOS
- Flake Templates (`nix flake init`)
- Flake Inputs: Git, Local Paths, Nixpkgs
- Flake Outputs: `packages` , `devShells` , `nixosConfigurations`

6.3 Flakes for Development

- Declarative Dev Environments (`devShells`)
- Language-Specific Templates (Python, Rust, Haskell, etc.)
- Pre-Commit Hooks & Linters
- CI/CD Integration with Flakes

6.4 Flakes for Deployment

- Deploying NixOS Systems with Flakes
- Flakes for Multi-Machine Deployments
- Flakes and Terraform/Ansible
- Flakes in CI/CD Pipelines (GitHub Actions, GitLab CI)

6.5 Flakes Best Practices

- Pinning Nixpkgs for Stability
- Updating Flake Inputs
- Flakes and Secrets Management
- Flakes in Teams: Collaboration Workflows

📌 Practical Lab 6: Nix Flakes in Action

- Create a flake for a Python project with dependencies.
 - Set up a `devShell` for a Rust project.
 - Deploy a NixOS configuration using flakes.
 - Integrate flakes with GitHub Actions.
-

Module 7: NixOS in Production

Objective: Learn to deploy, manage, and scale NixOS in production environments.

7.1 NixOS for Servers

- Web Servers (Nginx, Apache, Caddy)
- Database Servers (PostgreSQL, MySQL, Redis)
- Monitoring (Prometheus, Grafana, Loki)
- Logging (ELK Stack, Graylog)

7.2 NixOS for Kubernetes

- NixOS as a Kubernetes Node
- Kubernetes Cluster with NixOS
- NixOS and k3s
- Helm & NixOS Integration

7.3 NixOS and Infrastructure as Code (IaC)

- NixOS and Terraform
- NixOS and Ansible
- NixOS and Pulumi
- NixOS in GitOps Workflows (ArgoCD, Flux)

7.4 NixOS and CI/CD

- **GitHub Actions with Nix**
- **GitLab CI with Nix**
- **Nix in Jenkins**
- **Nix Caches for Faster Builds**

7.5 NixOS and Edge Computing

- **NixOS on Raspberry Pi & ARM Devices**
- **NixOS for IoT**
- **NixOS in Embedded Systems**
- **NixOS for Edge Servers**

7.6 NixOS and Security Hardening

- **CIS Benchmarks for NixOS**
- **NixOS and Zero Trust**
- **NixOS for Compliance (GDPR, HIPAA, SOC2)**
- **NixOS and Immutable Infrastructure**

Practical Lab 7: Production-Grade NixOS

- **Deploy a LAMP stack on NixOS.**
 - **Set up a Kubernetes cluster with NixOS nodes.**
 - **Automate NixOS deployments with Terraform.**
 - **Configure a CI/CD pipeline with Nix.**
-

Module 8: Troubleshooting & Debugging Nix/NixOS

Objective: Learn to debug and optimize Nix/NixOS systems.

8.1 Debugging Nix Builds

- **Build Failures: Logs & Error Messages**
- `nix-build` Debugging
- `nix-shell` Debugging
- Using `nix-instantiate` for Dry Runs

8.2 Debugging NixOS

- **System Boot Failures**
- **Service Debugging** (`journalctl` , `systemctl`)
- **Networking Issues** (`nmtui` , `ip` , `ss`)
- **Storage Issues** (`lsblk` , `blkid` , `mount`)

8.3 Performance Optimization

- **Nix Store Optimization**
- **Build Time Optimization** (Parallel Builds, Caching)
- **NixOS Boot Time Optimization**
- **Memory & CPU Profiling**

8.4 Nix/NixOS Anti-Patterns

- **Common Mistakes in Nix Expressions**
- **Avoiding Imperative Configurations**
- **Managing Large NixOS Configurations**
- **Handling Secrets Securely**

Practical Lab 8: Debugging Nix/NixOS

- Debug a failed Nix package build.
 - Troubleshoot a NixOS service that won't start.
 - Optimize a slow NixOS rebuild.
 - Fix a broken NixOS configuration.
-
-
-

Module 9: Nix/NixOS in the Real World

Objective: Learn from real-world use cases and community best practices.

9.1 Case Studies

- NixOS at Tweag (Haskell & Functional Programming)
- NixOS at IOHK (Cardano Blockchain)
- NixOS at Target (Retail & Cloud)
- NixOS in Academia (Reproducible Research)

9.2 Nix/NixOS Community & Resources

- NixOS Discourse & Matrix
- NixOS GitHub & GitLab
- NixOS Wiki & Documentation
- NixOS Conferences & Meetups

9.3 Nix/NixOS vs. Alternatives

- NixOS vs. Immutable Linux (Fedora Silverblue, Vanilla OS)
- Nix vs. Docker for Reproducibility
- NixOS vs. Ansible/Puppet/Chef for Configuration Management

9.4 Future of Nix/NixOS

- Upcoming Features in Nix/NixOS
- NixOS in the Cloud-Native Era
- NixOS and WebAssembly (WASM)
- NixOS and AI/ML Workloads

Practical Lab 9: Real-World Nix/NixOS Projects

- Replicate a real-world NixOS deployment (e.g., a web server).

- Contribute to a Nixpkgs package.
 - Set up a NixOS-based CI/CD pipeline for a project.
 - Compare NixOS with another immutable Linux distro.
-
-

Module 10: Capstone Project

Objective: Apply all learned concepts in a real-world project.

10.1 Project Ideas

- Deploy a Full-Stack Web App on NixOS
- Set Up a NixOS-Based Kubernetes Cluster
- Create a Reproducible Data Science Environment with Nix
- Build a NixOS-Based IoT Gateway
- Migrate an Existing System to NixOS

10.2 Project Requirements

- Declarative Configuration
 - Reproducible Builds
 - Automated Deployment
 - Monitoring & Logging
 - Documentation & Presentation
-

Capstone Lab: Build a Production-Grade NixOS System

- Choose a project (e.g., a web app, Kubernetes cluster, or IoT gateway).
 - Write a **complete NixOS configuration** (flakes recommended).
 - Deploy the system **locally or in the cloud**.
 - Document the process and present it.
-

Workbook & Practical Labs

To make this syllabus **actionable**, we'll compile a **workbook** with:

- 1. **Step-by-step labs** for each module.
 - 2. **Real-world scenarios** (e.g., "Deploy a NixOS web server with HTTPS").
 - 3. **Troubleshooting exercises** (e.g., "Debug a broken Nix derivation").
 - 4. **Community challenges** (e.g., "Package a new tool for Nixpkgs").
 - 5. **Quizzes & Assessments** to validate understanding.
-

Next Steps

- 1. **Review & Refine the Syllabus:** Should we add/remove any topics?
 - 2. **Prioritize Modules:** Which modules should we tackle first?
 - 3. **Gather Resources:** I'll compile **blogs, forums, and official docs** for each topic.
 - 4. **Build the Workbook:** We'll create **hands-on labs** for each module.
 - 5. **Iterate & Improve:** Refine based on your feedback and real-world testing.
-

◆ What do you think?

- Is this structure comprehensive enough?
- Should we dive deeper into any specific area (e.g., Kubernetes, security)?
- Would you like to start with a specific module for the workbook?

This is exactly the kind of course that I think is currently missing from the ecosystem.

After reviewing recent discussions across the Nix community, one recurring complaint appears over and over:

- Documentation is fragmented.
- Most tutorials stop at "here's my dotfiles."
- Almost nobody teaches **why Nix works internally**.
- There are very few production-focused courses comparable to RHCSA/RHCE, Linux Foundation, Kubernetes, or AWS certification tracks.
- There is no single curriculum that goes from **absolute beginner** → **Nix language expert** → **NixOS administrator** → **Nixpkgs contributor** → **Production Infrastructure Engineer**. ([NixOS Discourse](#))

I think we can build something significantly better.

Vision

Instead of "learning Nix"

The goal becomes

Become a Production Nix Infrastructure Engineer capable of designing, building, operating and debugging enterprise infrastructure using Nix and NixOS.

Think of it as the equivalent of

- RHCSA
- RHCE
- LFCS
- LFCE
- Kubernetes CKA
- Terraform Associate

combined into one curriculum centered around Nix.

Proposed Course

Professional Nix & NixOS Engineering

Total Duration

300–500 hours

Approximately

- 22 Modules
- 180+ Labs
- 15 Major Projects
- 4 Production Capstones

PART 0 – Linux Foundations

(Because many Nix users actually miss Linux fundamentals.)

Module 0

Linux Architecture

Topics

- Kernel

- Userspace
- init
- systemd
- boot process
- package managers
- permissions
- filesystem hierarchy
- networking
- processes
- cgroups
- namespaces
- capabilities
- PAM

Labs

- Build Linux VM
- Debug boot
- Analyze processes
- Create users
- Filesystem recovery

PART 1 – Philosophy

Module 1

Why Nix Exists

Topics

History

Problems with

apt

dnf

yum

pacman

brew

pip

cargo

npm

Dependency hell

Snowflake servers

Immutable infrastructure

Pure Functional Package Management

Reproducibility

Declarative Infrastructure

Garbage Collection

Content Addressing

Store Paths

Derivations

Outputs

References

Closures

Evaluation

Realization

Builders

Sandbox

Trust Model

Binary Cache

Substituters

Hydra

Cachix

Flakes

Current ecosystem

Future roadmap

Labs

Compare

Ubuntu

Fedora

Arch

Nix

Solve dependency conflicts

Measure reproducibility

PART 2 – Nix Language

This becomes almost a programming language course.

Module 2

Nix Syntax

Variables

Functions

Scopes

Recursion

let

with

inherit

Attribute Sets

Lists

Strings

Paths

Interpolation

Pattern Matching

Lazy Evaluation

Operators

Conditionals

Assertions

Builtins

Imports

Debugging

Tracing

Error Messages

Performance

Labs

60+

Mini parser

Configuration generators

Simple package descriptions

Expression evaluation

PART 3 – Functional Programming

Because many users struggle here.

Topics

Immutability

Purity

Lazy Evaluation

Closures

Higher Order Functions

Map

Filter

Fold

Composition

Currying

Fixed Points

Y combinator (simplified)

Recursion

Module System

PART 4 – Nix Package Manager

Everything

Store

GC

Profiles

Generations

Channels

CLI

New CLI

Legacy CLI

Search

Install

Develop

Run

Shell

Build

Copy

Verify

Diff

Store Optimization

Substituters

Signatures

Binary Cache

Labs

40+

PART 5 – Derivations

Deep dive

drv files

Builder

Environment

Inputs

Outputs

Dependency Graph

Closure

Store references

Fixed Output Derivations

CA Derivations

Incremental Builds

Projects

Write builders

Custom compiler

Package local software

PART 6 – nixpkgs

Massive section

Architecture

lib

pkgs

stdenv

Overlays

CallPackage

Override

OverrideAttrs

Package Sets

Cross Compilation

Platform Detection

Fetchers

Build Helpers

Languages

Python

Rust

Go

Node

Java

PHP

Ruby

Haskell

OCaml

C

C++

CUDA

Android

Projects

Package

50 different applications

PART 7 – Development Environments

devShell

mkShell

flake devShell

direnv

devenv

Lorri

FHS env

Docker integration

VSCode

Neovim

JetBrains

Projects

Python

Rust

Go

Node

Java

C++

Monorepo

PART 8 – Flakes

Complete coverage

Schema

Inputs

Outputs

Lock Files

Registry

Templates

Checks

Apps

Packages

DevShells

Configurations

Formatter

Hydra Jobs

CI

Cross-platform

Projects

Create reusable flakes

Enterprise template

PART 9 – Home Manager

Everything

User modules

Shell

Editors

Git

SSH

Fonts

GTK

KDE

GNOME

Hyprland

i3

Wayland

X11

Secrets

Project

Portable workstation

PART 10 – NixOS Module System

Most important section.

Module Evaluation

Options

Types

Merge Logic

mkOption

mkEnableOption

mkIf

mkMerge

mkDefault

mkForce

SpecialArgs

Imports

Custom Modules

Projects

Create modules

Custom services

Reusable infrastructure

PART 11 – NixOS Administration

Equivalent of RHCSA

Installation

Bootloader

Users

Storage

BTRFS

ZFS

LVM

Encryption

Networking

Firewall

SSH

Virtualization

Printing

Audio

Graphics

Drivers

Kernel

Firmware

Power

Services

Backups

Monitoring

Labs

80+

PART 12 – Services

NGINX

Apache

Postgres

MySQL

Redis

RabbitMQ

Kafka

Prometheus

Grafana

Loki

Traefik

Caddy

Vault

MinIO

Jellyfin

Nextcloud

Forgejo

GitLab

Mail

DNS

DHCP

VPN

WireGuard

Tailscale

Kubernetes

Docker

Podman

PART 13 – Secrets

Age

agenix

sops-nix

Vault

YubiKey

TPM

SSH CA

Rotation

Enterprise PKI

PART 14 – Containers

Docker

Podman

OCI

Image generation

nix2container

dockerTools

MicroVM

LXC

Incus

System Containers

PART 15 – Cloud

AWS

Azure

GCP

OCI

Hetzner

DigitalOcean

Scaleway

Proxmox

OpenStack

Terraform

OpenTofu

Deploy-RS

Colmena

Morph

NixOps

Projects

Deploy HA cluster

PART 16 – CI/CD

GitHub Actions

GitLab

Woodpecker

Buildkite

Hydra

Binary Cache

Cache Optimization

Remote Builders

PART 17 – Security

Enterprise security.

Secure Boot

TPM

LUKS

Measured Boot

SELinux comparison

AppArmor

systemd sandboxing

Namespaces

Capabilities

Firewalls

Kernel Hardening

Audit

Compliance

Supply Chain

Reproducible Builds

SBOM

Signing

Binary Trust

Secrets

This section should incorporate community hardening guidance and production practices that have become common in recent years, including Secure Boot, sops-nix, hardened networking, and sandboxing.

([Reddit](#))

PART 18 – Debugging

Evaluation Errors

Infinite Recursion

Store Corruption

GC Issues

Module Debugging

Performance

Tracing

REPL

Logs

Profiling

PART 19 – Performance

Remote Builders

Distributed Builds

Hydra

Cache

Nar Optimization

Closure Reduction

PART 20 – Production Architecture

Enterprise layout

Mono repo

Multi repo

Shared modules

Roles

Profiles

Machine Classes

Fleet Management

Thousands of servers

Hybrid Cloud

Disaster Recovery

Rollbacks

Canary

Blue/Green

PART 21 – Contributing

RFC

Packaging

Documentation

Testing

Hydra

CI

Maintainer workflow

Review process

Release process

PART 22 – Research Topics

Future of Nix

Content Addressed Store

Lix

Determinate Nix

RFCs

Performance research

Open problems

Academic work

Emerging tooling and ecosystem directions. ([arXiv](#))

Practical Workbook (Separate Volume)

I would not mix theory and labs. Instead, I'd create a dedicated workbook of roughly **200–250 hands-on labs** organized into progressive levels:

- **Level 1 – Fundamentals (40 labs):** install Nix, use the CLI, explore the store, write simple expressions, debug evaluation, create development shells.
- **Level 2 – NixOS Administration (50 labs):** users, networking, storage, systemd services, bootloader recovery, rollbacks, generations, hardware configuration.
- **Level 3 – Packaging & Modules (50 labs):** package applications in multiple languages, write overlays, create reusable modules, contribute fixes to nixpkgs.
- **Level 4 – Production Infrastructure (40 labs):** deploy multi-node environments, binary caches, remote builders, CI/CD, secrets management, monitoring, cloud provisioning.
- **Level 5 – Failure & Recovery (20+ labs):** intentionally break systems and recover them—corrupt configurations, dependency issues, failed deployments, rollback strategies, cache failures, networking outages.

The labs should culminate in **four capstone projects**:

1. Build a complete developer workstation from scratch.
2. Build and package a polyglot application stack (Go + Rust + Node + PostgreSQL).
3. Deploy a highly available NixOS production cluster with observability, secrets, CI/CD, and automated rollouts.

1. Design an enterprise-grade Nix repository capable of managing 100+ machines with reusable modules, testing, and release workflows.
-

Community Research Sources

Rather than relying only on official documentation, each module should be informed by patterns that repeatedly emerge in the community:

- Official `nix.dev` and NixOS manuals
- NixOS Wiki
- NixOS Discourse discussions
- RFCs and release notes
- High-quality GitHub repositories
- Determinate Systems engineering articles
- Reddit discussions (especially recurring beginner pain points and production lessons)
- Conference talks (NixCon)
- Blogs from long-term production users
- Academic literature where relevant

This approach reflects both the official guidance and the practical experience shared by users over the past several years. ([Nix](#))

One addition I'd make

If we're aiming for something on par with (or better than) RHCE, I would expand this into a **multi-volume curriculum** rather than a single book:

1. **Volume I:** Linux Foundations for Nix Engineers
2. **Volume II:** The Nix Language
3. **Volume III:** Nix Package Manager & nixpkgs
4. **Volume IV:** NixOS Administration

- i. **Volume V:** Production Infrastructure with NixOS
- ii. **Volume VI:** Packaging, Modules & Open Source Contributions
- iii. **Volume VII:** Enterprise Labs & Capstone Projects
- iv. **Volume VIII:** Troubleshooting, Performance & Internals
- v. **Volume IX:** Reference Manual (commands, options, module patterns, design patterns)

This would amount to roughly **2,500–4,000 pages of material** and **200–250 hands-on labs**, making it substantially more comprehensive than any existing structured Nix course I'm aware of. It would function as a long-term reference as well as a training program, rather than just a tutorial.

This comprehensive, enterprise-grade syllabus for **Nix and NixOS** is structured like a Red Hat Certified Architect (RHCA) track, moving systematically from fundamental language paradigms to enterprise-scale production infrastructure.

Enterprise Nix & NixOS Curriculum Architecture

Level 100: Foundations & Language Semantics

|

Level 200: Package Management & Standard Library (Nixpkgs)

|

Level 300: Modern Workspaces, Flakes & Dev Environments

|

Level 400: NixOS System Administration & Declarative OS Engine

|

Level 500: Advanced NixOS Modules & Secret Management

|
Level 600: Distributed Infrastructure, Deployment & Fleet
Orchestration
|
Level 700: Enterprise CI/CD, Binary Caching & Security Compliance
|
Level 800: Custom Derivations, Cross-Compilation & Internal
Machinery

Module 1: Functional Foundations & Nix Language Mechanics

Objective: Master the pure, lazy, functional domain-specific language that powers all Nix expressions.

1.1 Language Fundamentals

- **Functional Programming Constructs:** Immutability, pure functions, first-class functions, lexical scoping, and lazy evaluation.
- **Data Types & Data Structures:** Primitives (Strings, Integers, Floats, Booleans, Paths, Null) and Collections (Attribute Sets, Lists).
- **String Manipulation & Interpolation:** Indented strings (`''`), multiline text, path coercion, contextual escaping, and string context.
- **Scope & Binding:** `let ... in` expressions, `inherit` statements, function arguments (positional vs. attribute set destructuring), and default values.
- **Anti-Patterns & Code Health:** Avoiding `with` scope pollution, avoiding `rec` sets due to static analysis limitations, and avoiding legacy lookup paths (`<nixpkgs>`).

1.2 Evaluation & Built-ins

- **Evaluator Architecture:** How `nix-instantiate` converts expressions into Derivations (`.drv` files).
 - **Built-in Functions (`builtins`):** File system inspection (`builtins.readFile` , `builtins.readDir`), evaluation control, hashing, serialization (`fromJSON` , `toJSON`), and string evaluation.
 - **Lazy Evaluation Dynamics:** Thunks, call-by-need, infinite data structures, and debugging evaluation loops (infinite recursions).
-

Module 2: The Nix Store, Packaging & Nixpkgs Engine

Objective: Understand store-path calculation, derivation graphs, and package construction using `stdenv` .

2.1 The Nix Store Model

- **Store Layout:** `/nix/store/<hash>--<name-version>` structure and cryptographic content-addressing vs. input-addressing.
- **Derivation Anatomy:** `.drv` files as serialised build instructions; runtime dependencies vs. build-time dependencies.
- **Garbage Collection & Store Optimization:** GC roots, `nix-collect-garbage` , `nix-store --optimise` , and hardlinking identical files.

2.2 Derivation Construction & `stdenv`

- `stdenv.mkDerivation` Phase Lifecycle:

.. `unpackPhase` & `patchPhase`

!. `configurePhase` , `buildPhase` , and `checkPhase`

`}. installPhase & fixupPhase (patchelf, rpath rewriting, shearing binaries)`

- **Dependency Types:** `buildInputs`, `nativeBuildInputs`, `propagatedBuildInputs`, and `propagatedNativeBuildInputs`.
- **Packaging Patterns:** Wrapping binaries (`makeWrapper`), patching ELF interpreters (`patchelf`), overriding inputs via `.override` and `.overrideAttrs`.

2.3 Nixpkgs Architecture & Extensibility

- **Overlays:** Function signatures (`final: prev: { ... }`), overlay composition, avoiding infinite recursion in overlays.
 - **CallPackage Pattern:** Automatic dependency injection using `pkgs.callPackage`.
 - **Standard Library (`lib`):** Attribute set operations (`lib.attrsets`), list combinators (`lib.lists`), string utilities (`lib.strings`), and type predicates.
-

Module 3: Modern Nix Flakes & Declarative Workspaces

Objective: Transition from legacy Nix channels/NIX_PATH to pinned, reproducible Flake architectures.

3.1 Flake Architecture & Schema

- **Flake Inputs & Locks:** `flake.nix`, `flake.lock`, URI schemas (`github:`, `git+https://`, path-based inputs), input follow rules (`inputs.nixpkgs.follows`).
- **Standard Outputs Schema:** `packages`, `apps`, `devShells`, `nixosConfigurations`, `overlays`, `templates`, and `formatter`.

- **Flake Mechanics:** Hermetic evaluation, pure evaluation mode, and store isolation during evaluation.

3.2 Advanced Flake Frameworks & Tooling

- **Flake Modularization:** Structuring large repositories with `flake-parts` (replacing legacy `flake-utils`).
 - **Development Environments:**
 - Declarative shell setup with `pkgs.mkShell` and `nix develop`.
 - Integration with `direnv` + `nix-direnv` for automated workspace activation.
 - Specialized environment frameworks: `devenv.sh` vs. `devshell`.
-

Module 4: NixOS Core Architecture & System Administration

Objective: Master NixOS initialization, system-level declarations, and local administrative tasks.

4.1 NixOS Architecture Mechanics

- **Boot Sequence:** Linux kernel `initrd`, stage-1 and stage-2 boots, `systemd` generation switching, and `/run/current-system` linkage.
- **Atomic State Engine:** Generations, profile switching (`nixos-rebuild switch`), and transactional rollbacks (`--rollback`).
- **Hardware & Disk Declarations:**
 - Standard `hardware-configuration.nix` generation.
 - Modern declarative partitioning with **Disko** (GPT, LVM, LUKS, ZFS, Btrfs configurations).

4.2 System Services & Administration

- **System Services & Daemons:** Declarative `systemd.services`, timer units, user sandboxing, and runtime state management.
 - **User & Group Management:** Declarative users (`users.users.<name>`), SSH keys, and PAM integration.
 - **Networking & Security:** NetworkManager vs. `systemd-networkd`, declarative firewalls (`networking.firewall`), wireguard tunnels, and `sysctl` hardings.
-

Module 5: NixOS Module System, User Environments & Secrets

Objective: Build custom NixOS modules, manage user dotfiles with Home Manager, and secure production credentials.

5.1 The NixOS Module System Engine

- **Evaluation Framework:** `lib.evalModules`, module structure (`imports`, `options`, `config`).
- **Option Declarations:** `lib.mkOption`, types (`lib.types.str`, `lib.types.submodule`, `lib.types.listOf`), descriptions, and defaults.
- **Option Merging & Precedence:** `lib.mkIf`, `lib.mkMerge`, `lib.mkForce`, `lib.mkDefault`, and priority numbers.

5.2 User Environment Management (Home Manager)

- **Architecture:** Standalone installation vs. integrated NixOS module (`home-manager.nixosModules.home-manager`).
- **Declarative Dotfiles:** Managing `$XDG_CONFIG_HOME`, shell environments (Zsh, Fish, Bash), window managers/desktop environments, and application configs.

5.3 Production Secret Management

- **The Secret Problem in Pure Evaluation:** Why plain text in Nix store is globally readable (`/nix/store`).
 - **Secret Engines:**
 - **sops-nix:** Mozilla SOPS integration with age/GPG keys decrypted at boot time.
 - **agenix:** Age-encrypted secrets managed via SSH keys.
 - **Runtime Secret Injection:** Systemd `LoadCredential` and runtime tmpfs mounting patterns.
-

Module 6: Infrastructure Automation, Fleet Deployment & Bare-Metal Provisioning

Objective: Provision, deploy, and manage distributed enterprise server fleets across clouds and bare-metal.

6.1 Zero-Touch Provisioning

- **nixos-anywhere** : Remote automated installation over SSH onto running Linux systems (Hetzner, AWS, Bare-Metal).
- **Cloud-Init & Custom AMI/ISO Generation:** Building tailored Cloud-Init images, installer ISOs, and raw disk images via `nix build` `.#nixosConfigurations.host.config.system.build.sdImage` .

6.2 Fleet Orchestration & Remote Deployment

- **Orchestration Tools Comparison:**
- **Colmena:** Parallel, fast multi-node deployment engine.
- **deploy-rs:** Stateless, rust-based deployment tool.

- **nixos-rebuild target-host:** Native remote deployment over SSH.
 - **Cross-Architecture Builds:** Cross-compiling vs. remote builder daemons (`nix.buildMachines`) vs. QEMU binfmt emulation.
-

Module 7: Enterprise CI/CD, Binary Caches & Security Compliance

Objective: Operationalize Nix in enterprise build pipelines, establish binary caching infrastructure, and conduct security auditing.

7.1 Binary Caching Infrastructure

- **Public Caches:** Substituters, signing keys, and trusted public keys (`cache.nixos.org` , `nix-community.cachix.org`).
- **Enterprise Private Caches:**
- **Cachix:** SaaS managed cache deployment.
- **Attic / Harmonia / Nix-serve:** Self-hosted high-performance binary caches backed by S3/MinIO.
- **Cache Management:** Store path signing, key distribution, and eviction policies.

7.2 Enterprise CI/CD Integration

- **CI Architectures:** GitHub Actions, GitLab CI, and Hercules CI with Nix.
- **Optimized Builds:** `nix-eval-jobs` for parallel evaluation, caching layer extraction, and evaluation caching.
- **Container Build Pipelines:** Building minimal, zero-overhead OCI/Docker images using `pkgs.dockerTools.buildLayeredImage` (without

Docker daemons).

7.3 Vulnerability Management & Compliance

- **Vulnerability Scanning:** Scanning Nix derivations using `vulnix`, `grype`, or SBOM generators (`nix-sbom`).
 - **Software Bill of Materials (SBOM):** Generating CycloneDX/SPDX SBOMs directly from dependency graphs.
-

Module 8: Advanced Internal Machinery & Special Use Cases

Objective: Harness advanced Nix features for virtualization, language ecosystems, and core Nixpkgs contributions.

8.1 Language Framework Integration

- **Language-Specific Packaging Engines:**
- **Python:** `python3.withPackages`, `buildPythonPackage`, `poetry2nix`, `uv2nix`.
- **Rust:** `buildRustPackage`, `crane`, `naersk`.
- **Node.js/Go:** `buildGoModule`, `buildNpmPackage`, `dream2nix`.

8.2 Declarative Virtualization & MicroVMs

- **Lightweight Isolation:** `MicroVM.nix` for running hypervisor-isolated NixOS micro-VMs (Cloud-Hypervisor, QEMU, Firecracker).
 - **Systemd Containers:** Imperative and declarative `systemd-nspawn` container setups.
-

Enterprise Competency Matrix

Phase	Milestone	Core Tooling Mastered
Level 100–200	Nix Language & Derivations	<code>nix-instantiate</code> , <code>stdenv</code> , <code>patchelf</code> , <code>nix-repl</code>
Level 300–400	Reproducible Workspaces & Admin	Flakes, <code>flake-parts</code> , <code>direnv</code> , Disko, <code>nixos-rebuild</code>
Level 500–600	Modular Infrastructure & Fleet	Custom Modules, Home Manager, <code>sops-nix</code> , <code>nixos-anywhere</code> , Colmena
Level 700–800	Production Operations & Caching	Attic, <code>dockerTools</code> , <code>microvm.nix</code> , <code>poetry2nix</code> / <code>crane</code>

Suggested Next Steps

We can break down this curriculum into a hands-on **Practical Lab Workbook** featuring realistic scenarios (such as zero-touch cloud provisioning, configuring a secure Kubernetes/HA proxy cluster with `sops-nix` , or building private binary caches).

Would you like to start detailing the step-by-step practical labs and exercises for **Module 1 & 2** first, or would you prefer a specific enterprise scenario to build out?

To complement your study of NixOS, [NixOS in 2025 – The Most Advanced Linux OS Explained](#) provides a clear breakdown of NixOS concepts, declarative system management, and why it is widely adopted in modern DevOps workflows.